

PRACTICAL NO. 11

Docker Compose: Building and Orchestrating a Multi-Container Web Application

(Flask + Redis + Nginx Reverse Proxy Stack)

Aim	To install, configure, and use Docker Compose to define and run a multi-container application consisting of a web service, a cache/database service, and a reverse-proxy service.
Software Required	Docker Engine, Docker Compose (v2), a code editor, terminal / shell access
Platform	Ubuntu / Windows (WSL2) / macOS

1. Aim

To understand the concept of multi-container application orchestration using Docker Compose, and to design, build, run, scale, and tear down a real multi-service stack — a Flask web application, a Redis in-memory data store, and an Nginx reverse proxy — using a single `docker-compose.yml` file.

2. Objectives

- Understand the need for Docker Compose over standalone Docker commands.
- Learn the structure and syntax of a `docker-compose.yml` file (services, networks, volumes, ports).
- Build custom images as part of a Compose stack using the build directive.
- Connect multiple containers over a Compose-managed private network.
- Use Docker Compose commands to build, start, scale, monitor, and stop the application.

3. Theory

3.1 What is Docker Compose?

Docker Compose is a tool for defining and running multi-container Docker applications. Instead of starting each container manually with long `docker run` commands, Compose lets you describe the entire application stack — services, networks, and volumes — in a single declarative YAML file (`docker-compose.yml`). Running `docker compose up` then builds, (re)creates, starts, and connects all the containers for that stack in one command.

3.2 Why Multi-Container Applications?

Modern applications are rarely a single process. A typical web application needs a front-end/reverse proxy, an application/business-logic layer, and a data-storage or caching layer. Running each of these responsibilities in its own container follows the microservice and single-responsibility principles: each container can be scaled, updated, and restarted independently, while Compose manages the networking that lets them talk to each other by service name.

3.3 Architecture Used in this Practical

This practical builds a 3-tier containerized stack:

- `nginx` — reverse proxy exposed to the host on port 8080; forwards requests to the Flask app.
- `web` — a Flask application (custom image built from a `Dockerfile`) that increments and reads a hit counter.
- `redis` — an official Redis image used as the fast in-memory data store for the hit counter.

All three services are attached to a single Compose-managed bridge network so that they can reach each other using their service names (e.g., the Flask app connects to Redis simply via the hostname `"redis"`), without hard-coding any IP addresses.

4. Prerequisites

- Docker Engine installed and running (`docker --version`)
- Docker Compose v2 available (bundled with Docker Desktop, or the `docker-compose-plugin` on Linux)
- Basic familiarity with the Linux terminal and a text editor

```
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>docker --version
Docker version 29.4.1, build 055a478

D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>docker compose version
Docker Compose version v5.1.3
```

Fig. 4.1 — Verifying Docker and Docker Compose installation

5. Procedure

Step 1: Create the project directory structure shown below. Compose expects the docker-compose.yml at the project root; the Flask app and its Dockerfile live in web/, and the Nginx configuration lives in nginx/.



```
Terminal - Project Structure
$ tree practical9
practical9
├── docker-compose.yml
├── web
│   ├── Dockerfile
│   ├── requirements.txt
│   └── app.py
└── nginx
    └── default.conf

3 directories, 5 files
```

Fig. 5.1 — Project folder structure

Step 2: Create the Flask application file web/app.py. This app increments a counter stored in Redis on every request and returns the current count along with the container's hostname (useful later when we scale the web service).

web/app.py

```
from flask import Flask
from redis import Redis
import socket

app = Flask(__name__)
redis = Redis(host='redis', port=6379, decode_responses=True)

@app.route('/')
def hello():
    count = redis.incr('hits')
    hostname = socket.gethostname()
    return (f'Hello! This page has been viewed {count} times.\n'
            f'Served by container: {hostname}\n')

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Step 3: Create web/requirements.txt listing the Python dependencies needed by app.py.

web/requirements.txt

```
flask==3.0.3
redis==5.0.8
```

Step 4: Create web/Dockerfile to package the Flask application into a custom image.

web/Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Step 5: Create nginx/default.conf so that Nginx forwards all incoming traffic to the Flask service (using the Compose service name "web" as the hostname).

nginx/default.conf

```
server {
    listen 80;

    location / {
        proxy_pass          http://web:5000;
        proxy_set_header    Host $host;
        proxy_set_header     X-Real-IP $remote_addr;
    }
}
```

Step 6: Create the docker-compose.yml file at the project root. This single file defines all three services, how they are built/pulled, how they are networked, and which ports are published to the host.

docker-compose.yml

```
version: "3.9"

services:
  web:
    build: ./web
    expose:
      - "5000"
    depends_on:
      - redis
    networks:
      - appnet

  redis:
    image: redis:alpine
    networks:
      - appnet

  nginx:
    image: nginx:alpine
    ports:
      - "8080:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
```

```

depends_on:
  - web
networks:
  - appnet

networks:
  appnet:
    driver: bridge

```

Step 7: Build the images defined in the Compose file. Only the "web" service needs building (it has a build: directive); redis and nginx are pulled from Docker Hub.

```
docker compose build
```

```

#12 resolving provenance for metadata file
#12 DONE 0.1s
[+] build 1/1
✓Image lab_10-web Built

```

Fig. 5.2 — Output of docker compose build

Step 8: Start the entire stack in detached mode. Compose creates a dedicated bridge network, then starts redis, web, and nginx in the correct dependency order.

```
docker compose up -d
```

```

D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShivetaUdale\LAB_10>docker compose up -d
time="2026-09-18T16:58:21+05:30" level=warning msg="D:\\git and jira\\practical_Assignments_MCA_SY\\MCA2504150_ShivetaUdale\\LAB_10\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 25/25
✓Image redis:alpine      Pulled          16.8s
✓Image nginx:alpine      Pulled          14.8s
✓Network lab_10_appnet   Created         0.1s
✓Container lab_10-redis-1 Started         1.6s
✓Container lab_10-web-1   Started         1.5s
✓Container lab_10-nginx-1 Started         1.9s
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShivetaUdale\LAB_10>

```

Fig. 5.3 — Output of docker compose up -d

Step 9: Verify that all three containers are running and check their published ports.

```
docker compose ps
```

```

D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShivetaUdale\LAB_10>docker compose ps
time="2026-09-18T17:01:37+05:30" level=warning msg="D:\\git and jira\\practical_Assignments_MCA_SY\\MCA2504150_ShivetaUdale\\LAB_10\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
NAME                IMAGE             COMMAND                  SERVICE  CREATED      STATUS      PORTS
lab_10-nginx-1      nginx:alpine      "/docker-entrypoint..." nginx    2 minutes ago Up 2 minutes 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
lab_10-redis-1      redis:alpine      "docker-entrypoint.s..." redis    2 minutes ago Up 2 minutes 6379/tcp
lab_10-web-1        lab_10-web        "python app.py"         web      2 minutes ago Up 2 minutes 5000/tcp

```

Fig. 5.4 — Output of `docker compose ps`

Step 10: Test the application. Open a browser at `http://localhost:8080` or use `curl`. Refreshing / repeating the request should increment the hit counter, proving that the Flask container (web) is successfully talking to the Redis container over the Compose network, and that Nginx is correctly proxying to it.

```
curl http://localhost:8080/
```

```
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>curl http://localhost:8080/
Hello! This page has been viewed 1 times.
Served by container: 6e1fb533c354

D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>curl http://localhost:8080/
Hello! This page has been viewed 2 times.
Served by container: 6e1fb533c354

D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>curl http://localhost:8080/
Hello! This page has been viewed 3 times.
Served by container: 6e1fb533c354
```

Fig. 5.5 — Hit counter incrementing on each request, proxied through Nginx

Step 11: Inspect the logs of an individual service (useful for debugging).

```
docker compose logs web
```

```
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>docker compose logs web
time="2026-09-18T17:03:16+05:30" level=warning msg="D:\\git and jira\\practical_Assignments_MCA_SY\\MCA2504150_ShwetaUbale\\LAB_10\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
web-1 | * Serving Flask app 'app'
web-1 | * Debug mode: off
web-1 | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.19.0.3:5000
web-1 | Press CTRL+C to quit
web-1 | 172.19.0.4 - - [18/Sep/2026 11:32:29] "GET / HTTP/1.1" 200 -
web-1 | 172.19.0.4 - - [18/Sep/2026 11:32:48] "GET / HTTP/1.1" 200 -
web-1 | 172.19.0.4 - - [18/Sep/2026 11:32:51] "GET / HTTP/1.1" 200 -
```

Fig. 5.6 — Logs of the web service

Step 12: Scale the web service to run multiple replicas. Because Nginx proxies to the service name "web" (not a fixed container), and Compose load-balances across replicas by default, this demonstrates horizontal scaling with zero code changes.

```
docker compose up -d --scale web=3
```

```
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>docker compose up -d --scale web=3
time="2026-09-18T17:04:10+05:30" level=warning msg="D:\\git and jira\\practical_Assignments_MCA_SY\\MCA2504150_ShwetaUbale\\LAB_10\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 5/5
✓Container lab_10-redis-1 Running 0.0s
✓Container lab_10-web-1 Running 0.0s
✓Container lab_10-web-3 Started 1.2s
✓Container lab_10-web-2 Started 0.8s
✓Container lab_10-nginx-1 Running 0.0s
```

Fig. 5.7 — Scaling the web service to 3 replicas

Step 13: Stop and remove all containers, and the network created for this project, once testing is complete.

docker compose down

```
D:\git and jira\practical_Assignments_MCA_SY\MCA2504150_ShwetaUbale\LAB_10>docker compose down
time="2026-09-18T17:04:55+05:30" level=warning msg="D:\\git and jira\\practical_Assignments_MCA_SY\\MCA2504150_ShwetaUbale\\LAB_10\\docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] down 6/6
✔Container lab_10-nginx-1 Removed 0.5s
✔Container lab_10-web-1 Removed 1.8s
✔Container lab_10-web-3 Removed 1.7s
✔Container lab_10-web-2 Removed 1.7s
✔Container lab_10-redis-1 Removed 0.5s
✔Network lab_10_appnet Removed 0.3s
```

Fig. 5.8 — Output of docker compose down